



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

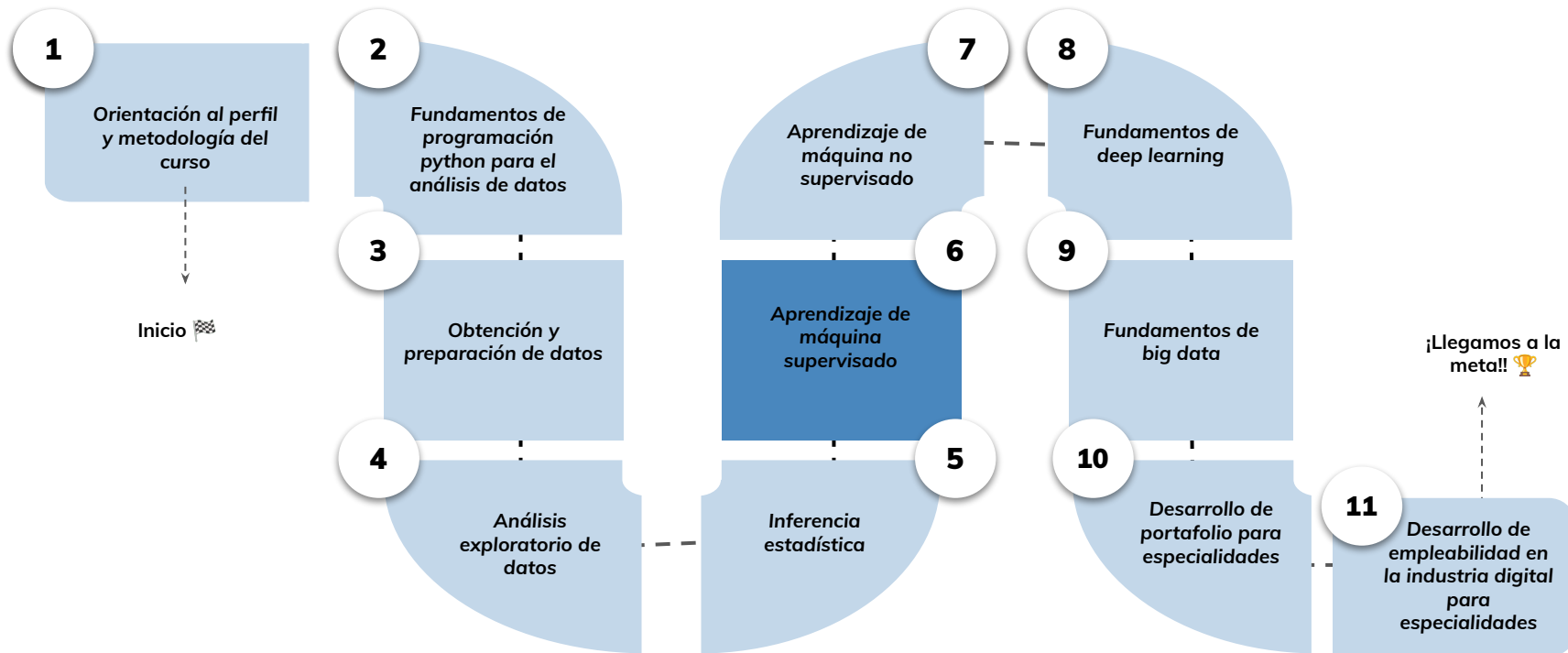


➤ Preprocesamiento y escalamiento de dato- Parte I

Aprendizaje Esperado 3: Aplicar técnicas para el preprocesamiento de datos en un modelamiento de aprendizaje de máquina utilizando las herramientas de python.

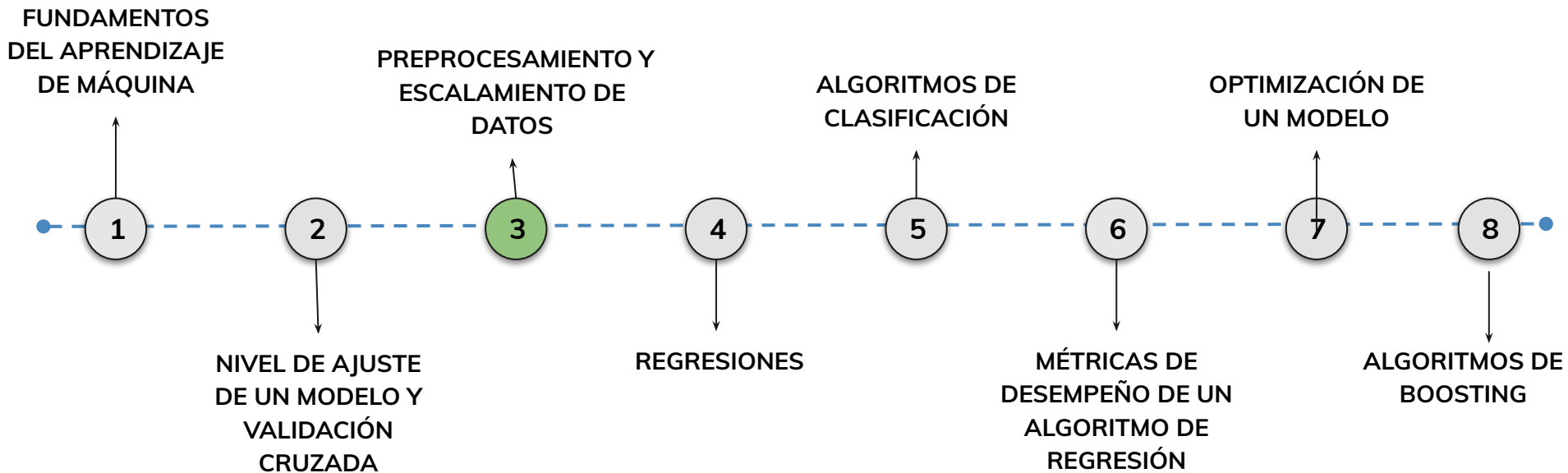
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Fundamentos del preprocesamiento de datos

Vamos a entender por qué los datos deben ser transformados antes de entrenar un modelo y qué problemas pueden aparecer si no lo hacemos.

Problemas comunes en los
datos

Valores faltantes y
duplicados

Variables categóricas y
numéricas
desbalanceadas

Técnicas de
preprocesamiento

Imputación,
codificación,
escalamiento

Implementación
con Scikit-Learn

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprendimos la importancia del preprocesamiento de datos
- Identificamos problemas comunes en datasets reales
- Aplicamos técnicas de imputación, codificación y escalamiento
- Transformamos variables categóricas correctamente
- Implementamos preprocesamiento con Scikit-Learn

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Cómo evaluar si un modelo está sobreajustado o subajustado
- Tipos de errores en modelos: sesgo, varianza y error irreducible
- Validación cruzada: qué es, para qué sirve y cómo aplicarla
- Técnicas K-Fold, ShuffleSplit y Leave-One-Out

Preprocesamiento y escalamiento de datos

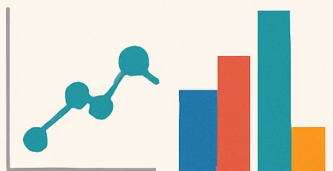
DATA PREPROCESSING

PREPROCESSING



MISSING VALUES

ENCODING



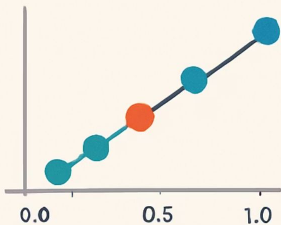
OUTLIERS

NORMALIZATION

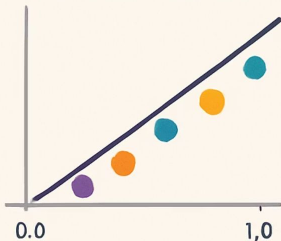


MISSINGIZATION

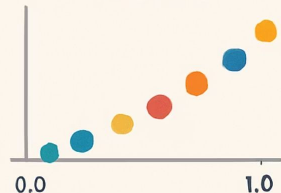
SCALING



STANDARDIZATION



SCALING

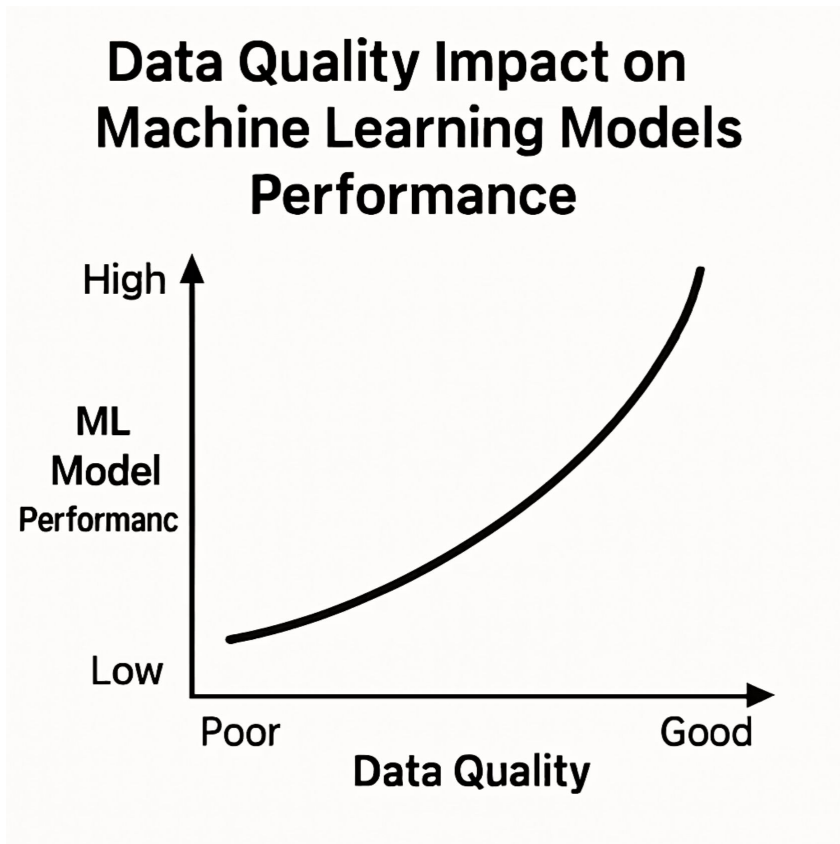


Preprocesamiento y Escalamiento de Datos

El preprocesamiento de datos es una de las etapas más importantes dentro del ciclo de vida de un proyecto de aprendizaje automático. Los datos en bruto, tal como se obtienen, suelen contener inconsistencias, formatos no estandarizados, valores faltantes y variables categóricas que necesitan ser transformadas para que los algoritmos puedan interpretarlas de manera adecuada.

Importancia del Preprocesamiento

La calidad y la preparación de los datos impactan directamente en el rendimiento y la precisión de los modelos predictivos. Un modelo de Machine Learning no podrá producir resultados fiables si la información utilizada para entrenarlo contiene errores, escalas desbalanceadas o variables mal codificadas. Por ello, el preprocesamiento y escalamiento de los datos constituyen un paso esencial.



skikit-



```
from sklearn.model_selection
import train_test_split

from sklearn.svm import SVC

X_train, X_test, y_train, y_test
= train_test_split(X, y, test_
    size=0.2)

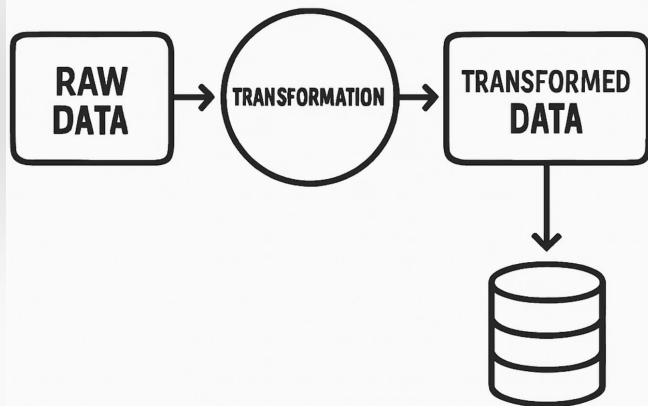
clf = SVC()

clf.fit(X_train, y_train)
```

Implementación con Scikit-Learn

Finalmente, aprenderás a implementar estos procesos utilizando la librería **Scikit-Learn**, una de las más utilizadas en el ecosistema Python para proyectos de Machine Learning. La combinación de teoría y práctica te permitirá dominar estas técnicas esenciales para la construcción de modelos eficientes y precisos.

RAW DATA TRANSFORMATION PROCESS



El Concepto de Preprocesamiento

Definición

El preprocesamiento de datos es un conjunto de técnicas y procedimientos destinados a preparar y transformar los datos en un formato adecuado para que los algoritmos de aprendizaje automático puedan trabajar con ellos de manera eficiente.

Problemas comunes

Los datos en su estado original suelen presentar múltiples problemas, como valores faltantes, formatos inconsistentes, ruido, duplicados y variables categóricas que los modelos no pueden interpretar directamente.

Importancia del Preprocesamiento

Naturaleza matemática

Una de las razones por las que el preprocesamiento es fundamental radica en que los algoritmos de Machine Learning son matemáticos por naturaleza. Por ello, requieren que los datos sean numéricos, estén libres de errores y sigan una estructura clara.

Calidad y limpieza

Otra motivación clave para realizar un buen preprocesamiento es la calidad y limpieza de los datos. Datos incompletos o mal formateados pueden generar sesgos en el modelo, disminuyendo su precisión y fiabilidad.

Aspectos Clave del Preprocesamiento

1

Transformación de variables categóricas

El preprocesamiento implica transformar las variables categóricas en variables numéricas. Este paso es esencial, ya que muchos algoritmos, como los basados en distancia (por ejemplo, K-Nearest Neighbors o SVM), no pueden operar con texto o etiquetas no numéricas.

2

Tratamiento de la escala

Otro aspecto clave es el tratamiento de la escala de las variables. Algunas técnicas de Machine Learning, especialmente las que calculan distancias, son sensibles a las diferencias de magnitud entre las variables. Por ello, la normalización o estandarización de los datos es un paso fundamental.

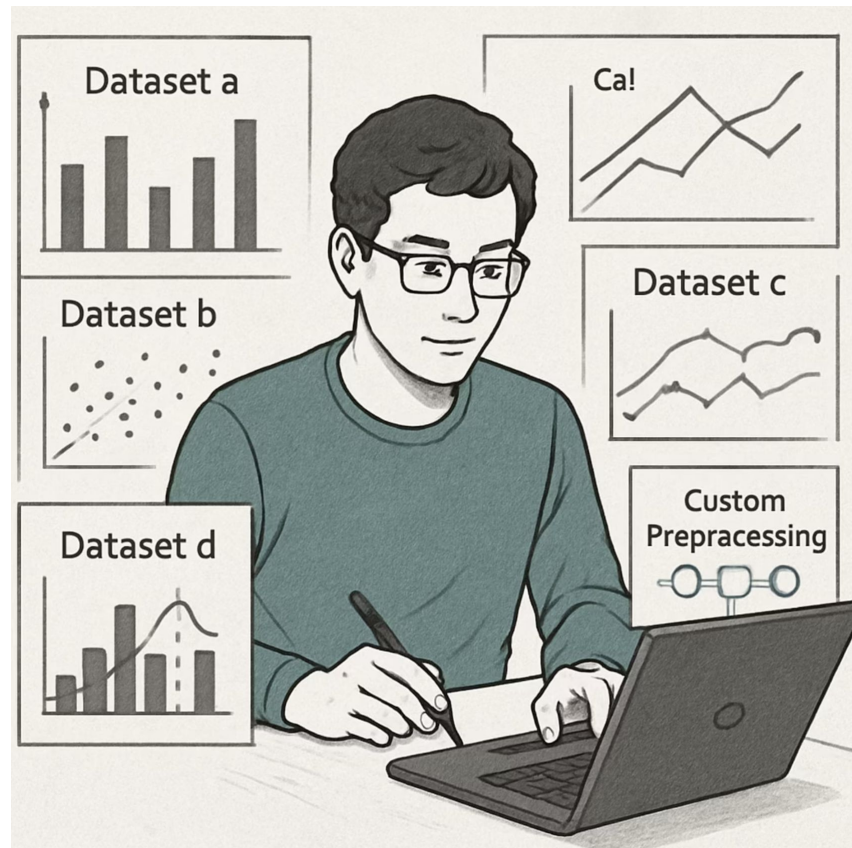
3

Detección de valores atípicos

El preprocesamiento incluye la detección y eliminación de valores atípicos, que pueden afectar la interpretación del modelo. La eliminación o transformación de estos valores contribuye a un mejor ajuste y evita resultados sesgados.

Personalización del Preprocesamiento

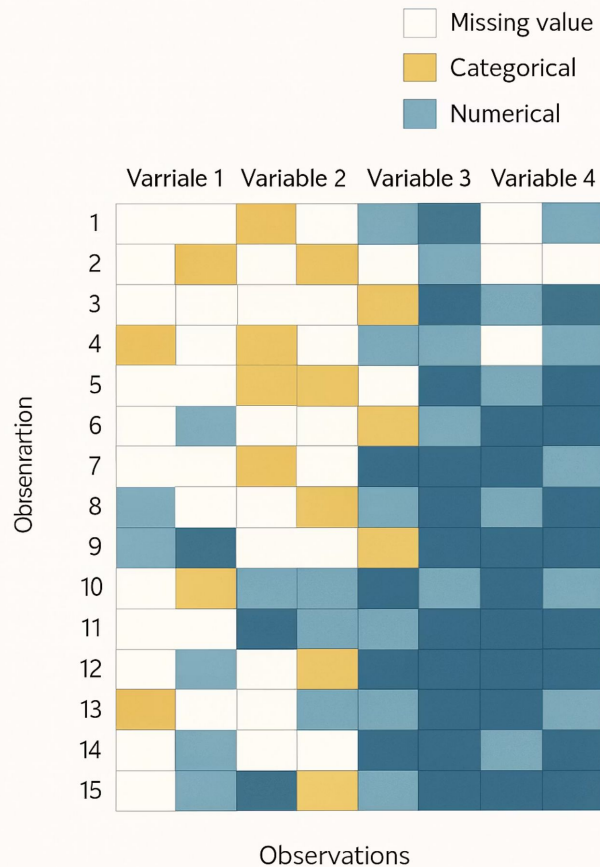
Por último, cabe destacar que el preprocesamiento no es un proceso único ni universal. Cada conjunto de datos requiere un análisis previo para identificar sus problemas específicos y aplicar las técnicas adecuadas. La correcta elección de las herramientas de preprocesamiento garantiza la integridad y utilidad de los datos en el proceso de modelado.



Ejemplo Práctico de Preprocesamiento

Supongamos que tenemos un conjunto de datos sobre clientes de un banco con las siguientes columnas:

ClienteID	Edad	Ciudad	Salario
1	25	Madrid	30000
2	45	Barcelona	50000
3	30	Sevilla	NaN



Problemas Comunes en los Datos

Variables categóricas

La variable **Ciudad** es categórica y necesita ser transformada a formato numérico.

Valores faltantes

Existe un valor faltante en la columna **Salario** que debe ser tratado.

Soluciones de Preprocesamiento



Imputación

Rellenar el valor faltante con la media o mediana del salario.



Codificación

Transformar la columna **Ciudad** a variables numéricas mediante codificación.

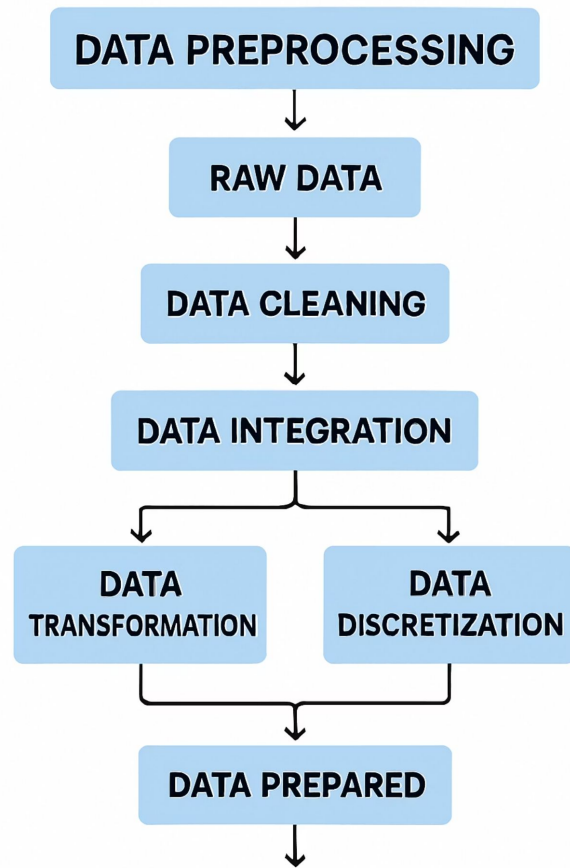


Escalamiento

Escalar las variables **Edad** y **Salario** para que estén en rangos similares.

Tareas Más Frecuentes de Preprocesamiento

El preprocesamiento de datos abarca una variedad de tareas cuya finalidad principal es garantizar que los datos sean adecuados para el entrenamiento y evaluación de modelos de aprendizaje automático. Estas tareas permiten corregir inconsistencias, reducir el ruido y transformar los datos en un formato que los algoritmos puedan procesar eficientemente.



Eliminación de Duplicados

Una de las tareas más comunes es la **eliminación de duplicados**. Los registros duplicados pueden distorsionar el análisis y provocar un sesgo en los modelos, ya que la información repetida puede ser interpretada como un patrón importante, cuando en realidad no aporta valor adicional.



Tratamiento de Valores Faltantes

Eliminación

Eliminar registros o columnas con valores faltantes cuando representan un porcentaje pequeño de los datos.

Imputación simple

Reemplazar valores faltantes con la media, mediana o moda de la columna.

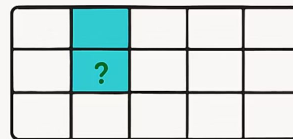
Imputación

avanzada como KNN Imputer que consideran la similitud entre observaciones para estimar valores faltantes.

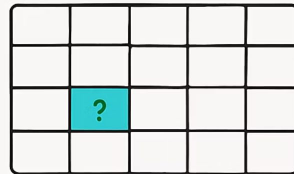
MISSING VALUES IMPUTATION TECHNIQUES



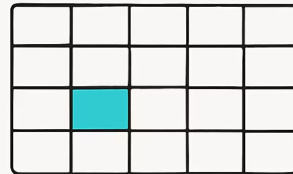
Deletion



Mean Imputation



Mean

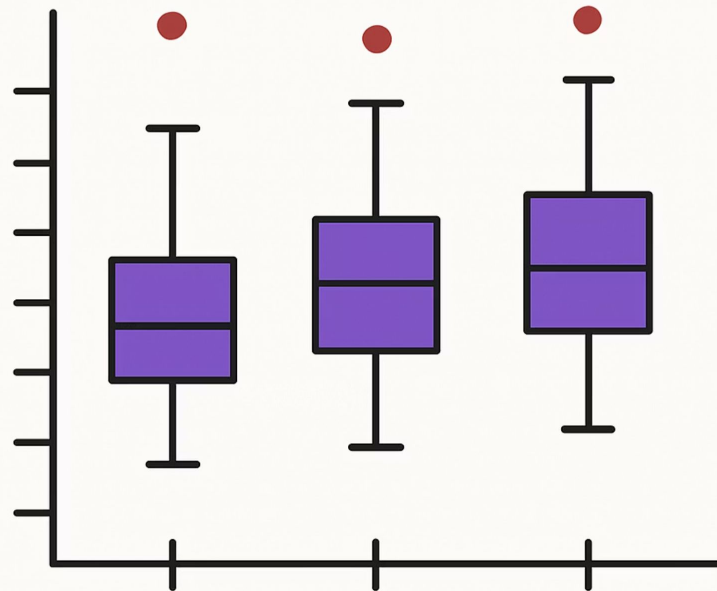


Most frequent

Tratamiento de Valores Atípicos

El **tratamiento de valores atípicos** también es esencial. Los outliers pueden tener un efecto desproporcionado sobre ciertos modelos, especialmente aquellos basados en distancias. Existen técnicas para detectar y gestionar estos valores, como el análisis de desviación estándar, los diagramas de caja y el método del rango intercuartílico.

Outlier Detection in Data Visualization



Transformación de Variables Categóricas

Otra tarea clave es la **transformación de variables categóricas**. Los algoritmos de Machine Learning requieren datos numéricos, por lo que es necesario convertir las variables de texto o categorías a un formato numérico mediante técnicas como **Label Encoding**, **One-Hot Encoding** o la creación de **variables dummy**.

CATEGORICAL VARIABLES TRANSFORMATION TECHNIQUES

ONE-HOT
ENCODING

ORDINAL
ENCODING

TARGET
ENCODING

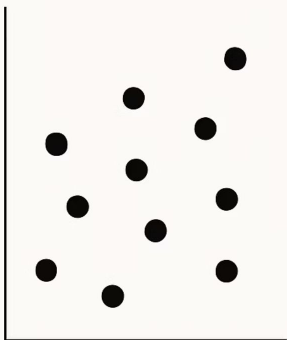
COUNT
ENCODING

BINARY
ENCODING

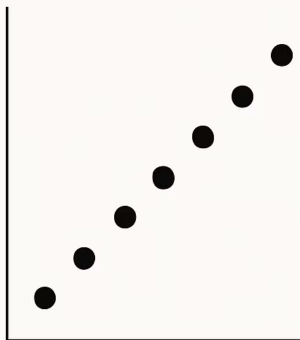
HASHING

Escalamiento de Variables

BEFORE



AFTER



VARIABLE SCALING

Importancia

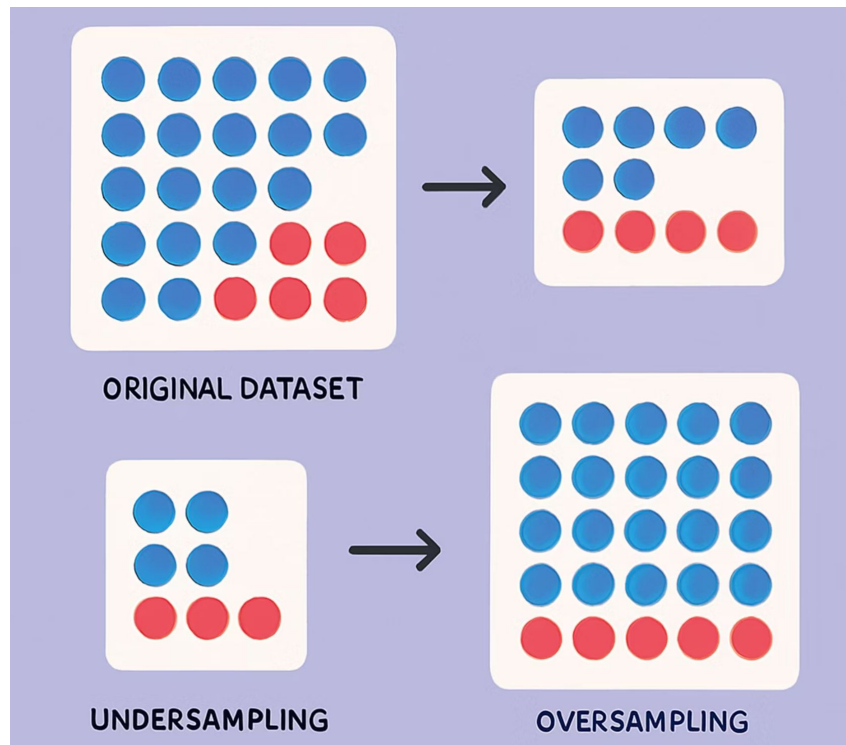
El **escalamiento de variables** es igualmente importante. Las variables numéricas pueden tener escalas muy diferentes, lo que afecta el comportamiento de algoritmos sensibles a la magnitud de los datos.

Técnicas

Es recomendable aplicar técnicas de normalización o estandarización para poner todas las variables en un rango comparable.

Balanceo de Clases

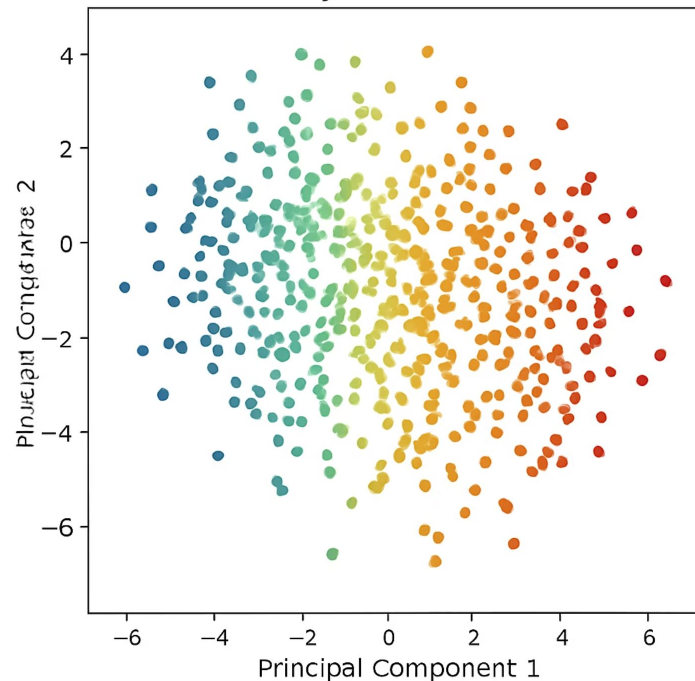
El **balanceo de clases** es otra tarea frecuente cuando se trabaja con problemas de clasificación. En muchos casos, las clases pueden estar desbalanceadas, provocando que el modelo favorezca a la clase mayoritaria. Técnicas como el sobremuestreo, submuestreo o el uso de algoritmos específicos permiten mitigar este problema.



Reducción de Dimensionalidad

Finalmente, la **reducción de dimensionalidad** puede ser necesaria cuando el conjunto de datos contiene un gran número de variables. Técnicas como PCA (Análisis de Componentes Principales) o selección de características ayudan a disminuir la complejidad y mejorar la interpretabilidad del modelo.

Dimensionality Reduction with PCA



Ejemplo Práctico de Preprocesamiento

Consideremos un conjunto de datos sobre pacientes con las siguientes variables:

PacienteID	Edad	Sexo	Presión	Diagnóstico
1	45	M	130	1
2	50	F	140	0
3	NaN	M	145	1
4	47	F	135	0

Tareas de Preprocesamiento Aplicadas

1

Imputación

El valor faltante en la columna Edad puede completarse con la media.

2

Transformación categórica

La columna Sexo puede codificarse (M=1, F=0).

3

Escalamiento

Las columnas Edad y Presión pueden ser normalizadas.

4

Eliminación de duplicados

Si existieran registros idénticos, deberían ser eliminados.

Label Encoding

Category
Encoded Value
1
2
3



One-Hot Encoding

Category		
Red	Green	
Red	1	0
Green	0	1
Blue	0	0

Label Encoding y One-Hot Encoding

En el preprocesamiento de datos, una de las tareas fundamentales es la transformación de variables categóricas a un formato numérico que los algoritmos de aprendizaje automático puedan procesar. Los algoritmos no son capaces de interpretar variables de tipo texto o categoría directamente, por lo que es necesario convertirlas en valores numéricos. Dos de las técnicas más utilizadas para este propósito son **Label Encoding** y **One-Hot Encoding**.

Label Encoding

Label Encoding consiste en asignar un valor numérico único a cada categoría de una variable. Por ejemplo, si tenemos una variable "Color" con las categorías Rojo, Azul y Verde, Label Encoding las transformaría en 0, 1 y 2, respectivamente.

Esta técnica es muy sencilla y rápida de implementar, pero tiene una limitación importante: introduce un orden artificial entre las categorías, lo que puede confundir a ciertos algoritmos.

Categorical
Data

Class A
Class B
Class C
Class A

Label
Encoding



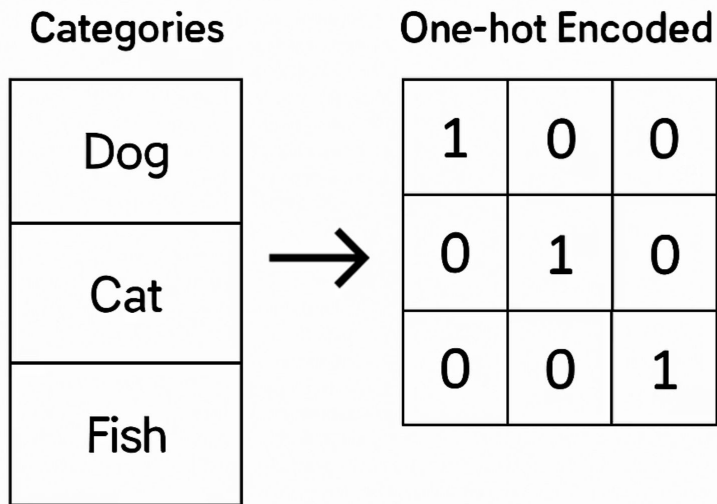
Numerical
Labels

0
1
2
0

One-Hot Encoding

Por otro lado, **One-Hot Encoding** genera una nueva columna para cada categoría posible y asigna un 1 o un 0 según si la observación pertenece a esa categoría. Por ejemplo, para la variable "Color" mencionada antes, One-Hot Encoding crearía tres columnas: Color_Rojo, Color_Azul y Color_Verde. Una observación con valor "Azul" tendría los valores [0,1,0] en estas columnas.

One-hot Encoding Transformation Process



Comparación entre Técnicas de Codificación

Label Encoding

- Ventaja: Mantiene el conjunto de datos compacto
- Ventaja: Funciona bien con algoritmos basados en árboles
- Desventaja: Introduce un orden artificial entre categorías

One-Hot Encoding

- Ventaja: No introduce orden entre categorías
- Ventaja: Ideal para modelos basados en distancia
- Desventaja: Aumenta la dimensionalidad del conjunto de datos

Consideraciones Adicionales



Dimensionalidad

Una desventaja de One-Hot Encoding es que puede aumentar significativamente la dimensionalidad del conjunto de datos cuando la variable categórica tiene muchas categorías. Esto puede impactar el rendimiento del modelo y aumentar los requerimientos computacionales.



Variables binarias

En ciertos casos, como cuando se trabaja con variables binarias (dos categorías), Label Encoding y One-Hot Encoding producirán el mismo efecto, ya que una única columna codificada será suficiente para representar la información.

Ejemplo Práctico de Label Encoding

Consideremos el siguiente conjunto de datos:

ID	Ciudad
1	Madrid
2	Sevilla
3	Madrid

```
from sklearn.preprocessing import LabelEncoder

ciudades = ['Madrid', 'Sevilla', 'Madrid', 'Barcelona']
encoder = LabelEncoder()
ciudades_codificadas = encoder.fit_transform(ciudades)

print(ciudades_codificadas)
```

Aplicando Label Encoding obtenemos:

```
[1 2 1 0]
```

Interpretación: Barcelona = 0, Madrid = 1, Sevilla = 2.

Ejemplo Práctico de One-Hot Encoding

Utilizando el mismo conjunto de datos:

ID	Ciudad
1	Madrid
2	Sevilla
3	Madrid

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

ciudades = np.array(['Madrid', 'Sevilla', 'Madrid', 'Barcelona']).reshape(-1,1)
encoder = OneHotEncoder(sparse=False)
ciudades_ohe = encoder.fit_transform(ciudades)

print(ciudades_ohe)
```

Aplicando One-Hot Encoding obtenemos:

```
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]
```

Cada fila representa la ciudad en formato One-Hot Encoding. Por ejemplo, la fila [0. 1. 0.] indica la categoría "Madrid".

Variables Dummy

Las **variables dummy** son una técnica utilizada para convertir variables categóricas en variables numéricas, facilitando su uso en modelos estadísticos y de aprendizaje automático. El concepto de variables dummy es muy similar al de One-Hot Encoding, ya que ambos procedimientos buscan representar categorías mediante valores binarios.

Categorical Variable		Dummy Variables		
Observation	Category	Observation	A	C
1	A	1	1	0
2	B	2	1	1
3	C	3	0	0
4	C	4	0	1
5	A	5	1	0
6	A	0	0	0
7	C	1	0	1
8	B	0	0	1
8	B	8	0	0

Dummy Variables Creation Process

Creación de Variables Dummy

Cuando se crea una variable dummy, se genera una nueva columna por cada categoría de la variable original. Cada una de estas columnas toma el valor 1 si la observación pertenece a esa categoría y 0 en caso contrario. Por ejemplo, si la variable original es "País" con las categorías "España", "Francia" e "Italia", el procedimiento dummy creará tres columnas: País_España, País_Francia y País_Italia.

Dummy Variables Example with Country Categories

Country	USA	Canada	Germany
USA	1	0	0
Canada	0	0	0
Germany	1	0	1
Canada	0	0	0

Multicolinealidad en Variables Dummy

Problema

Una diferencia importante entre One-Hot Encoding y la creación de variables dummy es que, en algunos contextos, se recomienda eliminar una de las columnas generadas para evitar la **multicolinealidad**. Este fenómeno ocurre cuando una de las variables puede ser predicha perfectamente a partir de las otras, afectando negativamente a los modelos lineales.

Solución

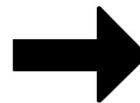
Las variables dummy son ampliamente utilizadas en modelos estadísticos tradicionales, como la regresión lineal y la regresión logística, donde la multicolinealidad puede afectar los coeficientes del modelo. En estos casos, eliminar una columna dummy evita redundancia y garantiza que las variables sean independientes.

Variables Dummy para Categorías Binarias

Otra característica de las variables dummy es que, cuando las categorías son binarias, el procedimiento genera una única columna. Por ejemplo, si la variable "Sexo" contiene las categorías "M" y "F", una sola variable dummy sería suficiente para representar la información.

Binary
Categorical
Variable

Status
Yes
No
Yes
No



Dummy
Variables

Status_No	Status_Yes
0	0
1	0
0	1
1	0

```
def get_dummies(df):

    let dummies =
        for col in df.columns:
            if df[col].dtype == 'object':
                for val in
                    df[col].unique():

                        dummy_col = (df[col]
                                     == val).astype(int)
                        dummies.append(dummy_col)
            else:
                dummies.append(df[col])

    else:
        dummies.append(df[col])

    return pd.concat(dummies, axis=1)
```

Implementación de Variables Dummy

En la práctica, muchas librerías de análisis de datos, como Pandas, ofrecen funciones específicas para la creación automática de variables dummy. Esto simplifica el proceso y evita errores manuales durante la codificación.

Las variables dummy también son útiles cuando se trabaja con modelos interpretables, ya que permiten identificar claramente el impacto de cada categoría sobre la variable objetivo. Además, son compatibles con algoritmos que requieren entradas numéricas y evitan el problema del orden artificial introducido por Label Encoding.

Ejemplo Práctico de Variables Dummy

Supongamos que tenemos el siguiente conjunto de datos:

ID	País
1	España
2	Francia
3	Italia
4	España

Aplicando variables dummy obtenemos:

ID	País_Francia	País_Italia
1	0	0
2	1	0
3	0	1
4	0	0

En este caso, la categoría "España" fue eliminada para evitar multicolinealidad. Si un registro tiene Francia=0 e Italia=0, se asume que pertenece a España.

```
import pandas as pd

datos = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'País': ['España', 'Francia', 'Italia', 'España']
})

# Crear variables dummy
variables_dummy = pd.get_dummies(datos['País'], drop_first=True)

# Unir con el DataFrame original
datos = pd.concat([datos, variables_dummy], axis=1)

print(datos)
```

Live Coding

¿En qué consistirá la Demo?

Vamos a trabajar con un dataset con múltiples problemas típicos de preprocesamiento: valores faltantes, variables categóricas y escalas distintas. Vamos a transformarlo y prepararlo para ser usado por un modelo.

1. Cargar un dataset tabular
2. Detectar valores nulos y duplicados
3. Aplicar imputación con media/mediana
4. Codificar variables categóricas con One-Hot
5. Escalar variables numéricas con StandardScaler
6. Mostrar el resultado final
7. Comparar con el dataset original



Momento:

Time-out!



5 -10 min.





Ejercicio

Pre procesando datos de clientes

Pre procesando datos de clientes

Contexto: 🙌

Estás trabajando con un dataset de clientes bancarios que será utilizado para predecir si un cliente aceptará una oferta de tarjeta de crédito. Antes de entrenar un modelo, necesitas dejar los datos listos para que los algoritmos puedan usarlos correctamente.

Consigna: 📝

Aplice técnicas de preprocesamiento para dejar el dataset en condiciones de ser usado por un modelo de aprendizaje automático.

Paso a paso: ⚙️

1. Cargue el dataset clientes.csv
2. Verifique si hay valores faltantes y decidir cómo imputarlos
3. Identifique variables categóricas y codificarlas con One-Hot
4. Escale las variables numéricas
5. Guarde el dataset transformado como clientes_preprocesado.csv
6. Compartir insights con el grupo:
 - ¿Qué columnas fueron transformadas?
 - ¿Cómo cambió el shape del dataset?
 - ¿Hay columnas que podrían eliminarse?

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Entendimos la importancia del preprocesamiento en Machine Learning
- ✓ Identificamos problemas comunes como nulos, outliers y escalas desiguales
- ✓ Aplicamos técnicas de imputación, codificación y escalamiento
- ✓ Usamos Scikit-Learn para transformar datos reales
- ✓ Dejamos listo un dataset para modelado automático



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

